

RAG-системы Векторные базы данных

Лекция 14

Кудин Степан, старший математик-разработчик

Андрей Кривой, руководитель группы внутренних ML-продуктов

Про нас

- Работаем в Озоне в команде Внутренних ML-продуктов с 2025 г.
- Степан много лет работал в Лаборатории ИИ Сбера, где занимался компьютерным зрением в медицине
- Андрей до этого много лет работал в VK, занимался качеством поиска во ВКонтакте, Одноклассниках, Дзене и других проектах VK
- В Озоне мы поднимаем, оптимизируем и обучаем LLMки
- А еще решаем прикладные задачи: делаем чатботы и RAG-системы, кодовые ассистенты, распознаем документы, занимаемся видеоаналитикой, ...
- Помогаем другим командам применять ИИ в своих проектах



Структура курса

Сегодня заканчиваем блок дополнительных тем

1. Что такое "большие данные"?
2. Архитектура распределенных систем
3. Введение в Apache Spark.
4. Основные абстракции данных

5. ML на больших данных: классические алгоритмы
6. ML на больших данных: нейросети

7. Оптимизации при обучении нейросетей
8. Data-parallel обучение нейросетей
9. Обучение больших моделей

10. Софт для инференса нейросетей
11. LLM inference
12. Эффективный инференс моделей

13. MLOps
14. RAG-системы. Векторные базы данных



01 { Задача

Какую задачу мы хотим решить?

- Мы хотим создать такую программу, которая будет давать правильные ответы на наши вопросы по нашим данным.
- Кажется, что LLM могут нам помочь довольно быстро сделать такую программу.
- Это действительно так, но есть нюансы, главный из которых – LLM в чистом виде скорее всего не сможет отвечать на наши вопросы верно.

Почему любая LLM не может отвечать на любые вопросы?

- Знания LLM заканчиваются на дате, по которую были собраны обучающие данные. Что было дальше она точно не знает.
- LLM не обучалась на приватных данных (если их, конечно, каким-либо образом не слили в датасет). Соответственно ничего не знает о приватных документах, по которым вы хотите задавать вопросы.

Что с этим можно сделать?

- Можно дообучить LLM на своих данных. Этот подход будет работать, но он сложен, дорог и не гибко.
- Можно в промпте передавать контекст, достаточный для ответа на вопрос. Такой подход тоже работает, но проще в реализации и более гибкий. Это и есть RAG.

02 { Что такое RAG

Что такое RAG?

- RAG (Retrieval-Augmented Generation) – это технология, которая улучшает работу LLM за счёт предоставления ей данных из внешних баз знаний.
- Подход сочетает в себе поиск релевантной информации (retrieval) с генерацией текста (generation), позволяя LLM верно ответить на запрос пользователя, не имея нужных знаний в своих весах.

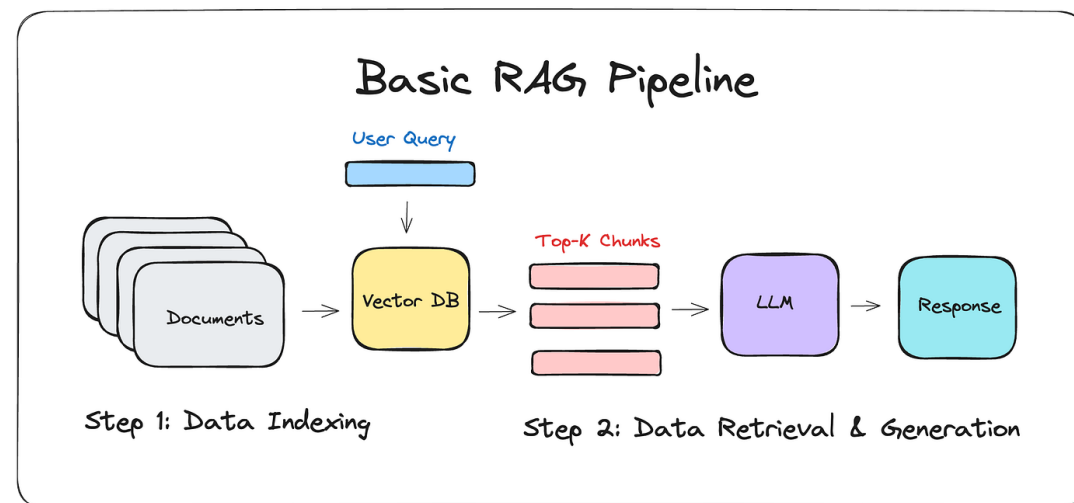
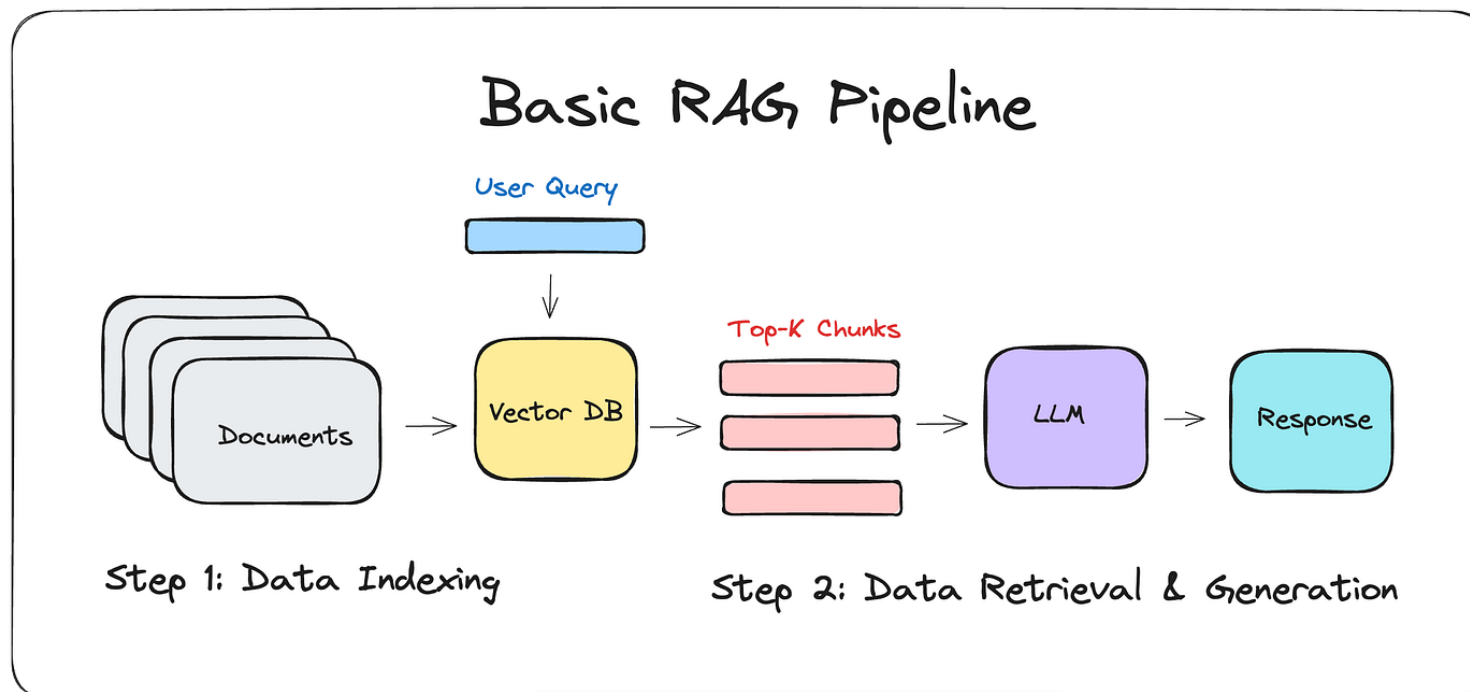


Схема работы RAG

- Добавление документов в векторную БД (выполняется один раз)
- Обработка запроса пользователя
- Поиск
- Генерация ответа



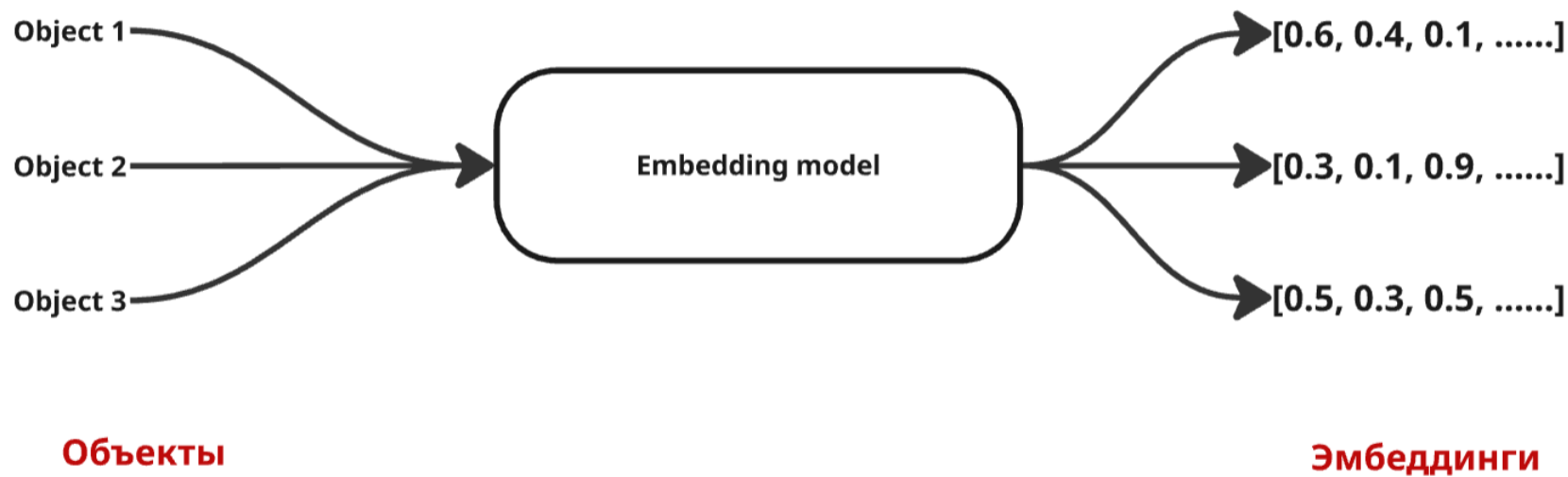
03



Эмбе́ддинги и векторные
базы данных

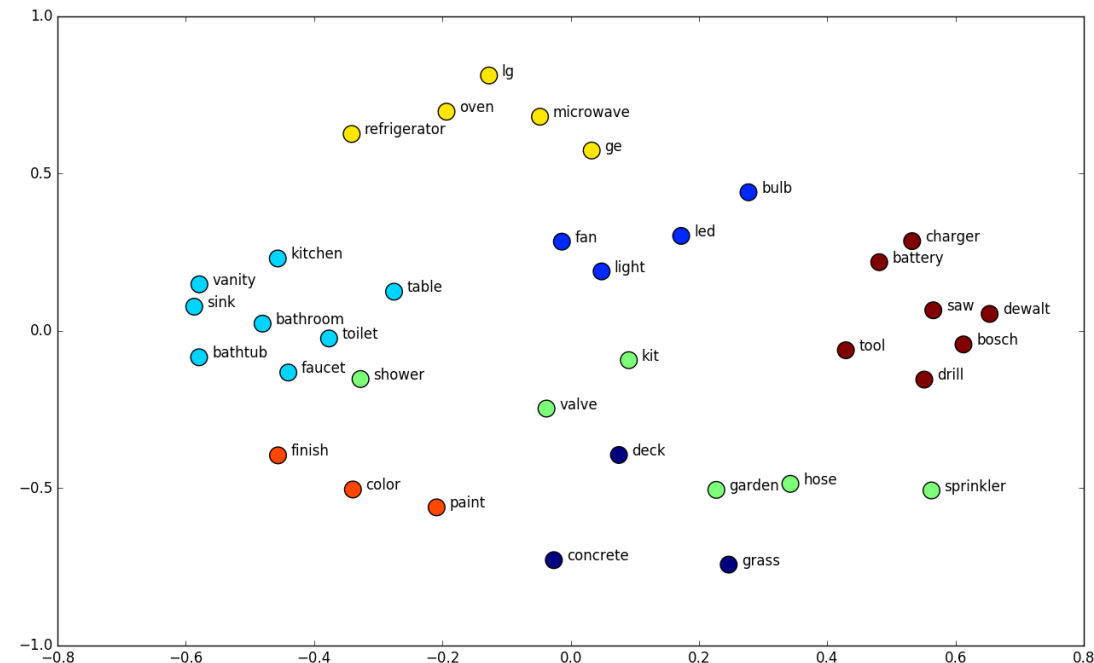
Что такое эмбединг?

- Эмбединг (embeddings) – это вектор в некотором векторном пространстве $\mathbb{R}^n$, в который отображается некий объект.
- Объектами могут быть: изображения, текст, звук и т.д.
- Функцией отображения обычно выступает нейронная сеть, но не обязательно.
- Используются для численного представления объектов для дальнейшей работы с ними.



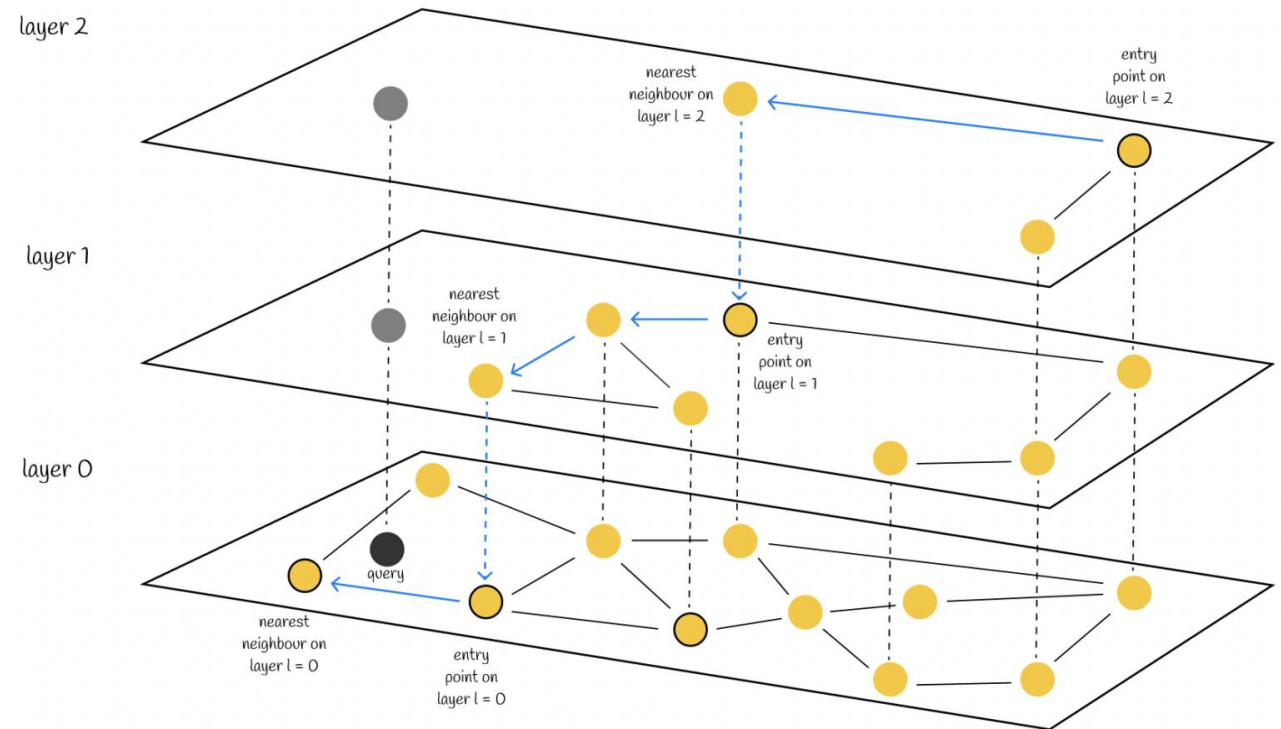
Зачем в RAG-ах эмбединги?

- В RAGах используются такие отображения, которые отображают семантически близкие тексты в близкие в векторном пространстве вектора.
- Это позволяет искать информацию по запросу пользователя, т.к. текст запроса и ответы на него обычно находятся рядом в векторном пространстве, а не релевантные для запроса тексты, находятся от него достаточно далеко.
- Такой подход удобен тем, что мы ищем именно по смыслу, а не по ключевым словам.



Что такое векторная база данных?

- Векторная база данных – это специализированное хранилище данных для быстрого поиска близких векторов.
- Почему нельзя использовать обычную реляционную SQL БД? Потому что она ищет по словам.
- Зачем она вообще нужна? Из-за очень быстрого поиска векторов, близких к запросу. Когда у вас 10, 100, 1 000 векторов это не так критично и можно решать задачу перебором. Но когда у вас много запросов и/или много векторов (10 000 и больше) скорость поиска становится критичной.
- Для быстрого поиска такие БД используют алгоритмы HNSW, IVF и им подобные.



Как работают векторные базы данных?

- Добавление данных. Вы передаёте в базу данных следующее:
 - Исходный объект (опционально).
 - Эмбеддинг.
 - Метаданные, например дату, автора, теги и так далее (опционально).
- Индексация. На этом этапе база данных строит внутреннюю структуру для ускорения поиска.
- Поиск. Вы передаёте эмбеддинг запроса и база данных возвращает вам топ-N точек в векторном пространстве со всем дополнительными данными, привязанными к ним.
- Фильтрация (присутствует не во всех БД). Можно фильтровать точки по метаданным, например по источнику.

04



Небольшой обзор некоторых векторных БД

Chroma

- Встраиваемая векторная БД, написанная на Python.
- Хорошо подходит для прототипов и небольших приложений.
- Open-source.
- Сайт: <https://trychroma.com>



Chroma

Плюсы

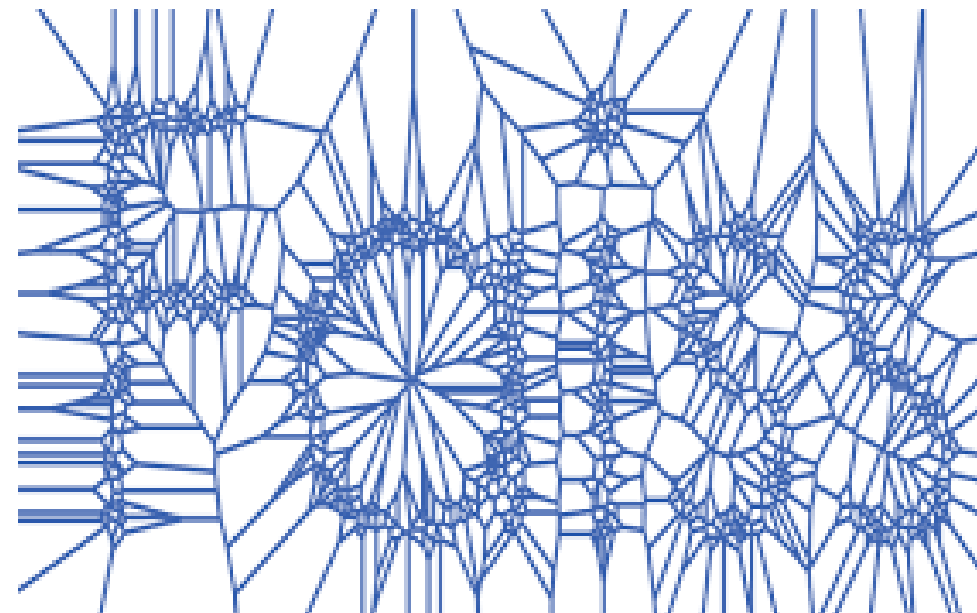
- Не требуется отдельный сервер.
- Данные можно хранить как в памяти, так и на диске.
- Очень просто завести в проекте.

Минусы

- Не масштабируется.
- Не подходит для нагруженного продакшена.
- Ограниченная поддержка фильтрации по метаданным.

FAISS

- Библиотека от Meta для ускоренного поиска по векторам.
- Хорошо подходит для батч-обработки.
- Open-source.
- Сайт: <https://faiss.ai>



Плюсы

- Высокая скорость поиска.
- Прекрасно подходит для статических наборов данных (когда не требуется частое обновление).
- Не требуется отдельный сервер.

Минусы

- Не поддерживает динамические обновления. После добавления данных нужно пересоздавать индекс.
- Не работает с метаданными.
- Требуется ручное управление индексами.
- Это библиотека, а не полноценная база данных.

Qdrant

- Высокопроизводительная векторная база данных написанная на Rust с HTTP/gRPC API.
- Разрабатывается для серьёзного продакшена.
- Хороший выбор для продакшен сервиса с динамическими данными, фильтрацией, RAG.
- Open-source.
- Сайт: <https://qdrant.tech>



Qdrant

Плюсы

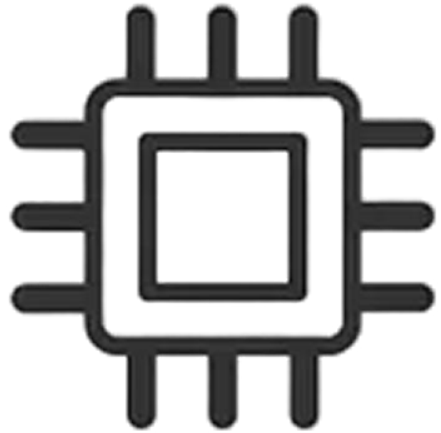
- Бесплатна в локальной инсталляции.
- Высокая скорость поиска.
- Есть полноценные CRUD-операции.
- Поддержка сложных фильтров.
- Готова к масштабированию, мониторингу и отказоустойчивости.

Минусы

- Требуется отдельный процесс.
- Сложнее в настройке, чем Chroma.

Что есть ещё?

- pgvector – расширение к PostgreSQL, которое добавляет тип данных vector и операторы поиска по расстоянию.
- Weaviate – аналог Qdrant.



pgvector



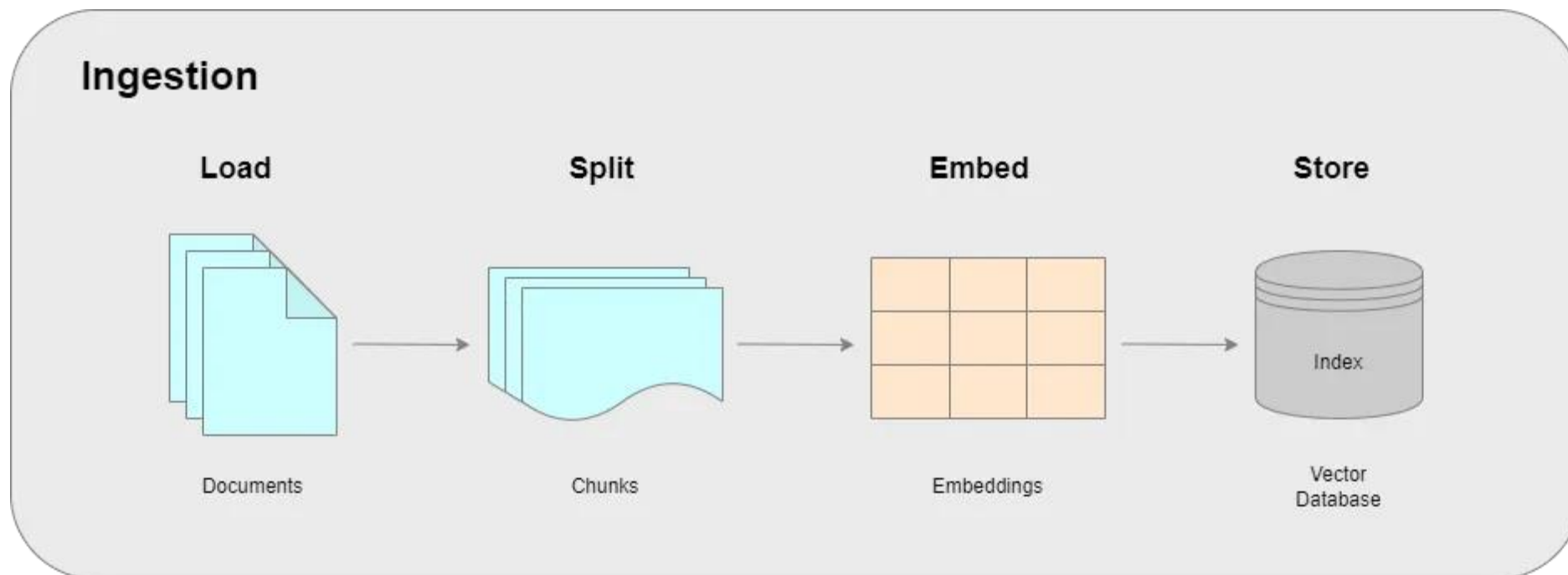
05



Добавление документов в
векторную БД

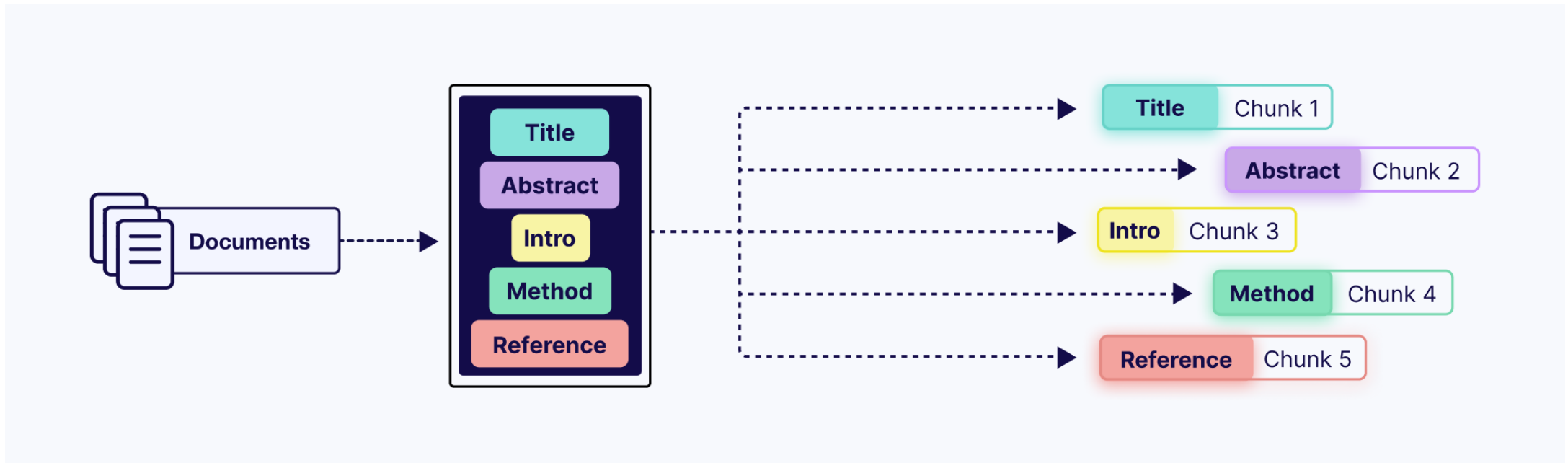
Общая схема работы

- Получаем нужные документы
- Разбиваем документы на чанки (кусочки)
- Преобразуем каждый чанк в эмбединг с помощью специальной модели
- Сохраняем чанки вместе с соответствующими векторами в специальной векторной базе данных



Чанкинг (chunking)

- Чанк – это кусок текста, который хранится как отдельная запись.
- Чанкинг – это процесс разбиения текста на чанки.
- Чанкинг можно производить по разному. Самый простой – разбить текст на кусочки фиксированной длины.



Зачем нужен чанкинг? Почему не загружают документ целиком?

- В длинном тексте смысл размывается, из-за этого ухудшается семантический поиск.
- LLM имеет ограничение на контекст, есть риск из него выйти.
- При большом контексте LLM чаще ошибается, потому что размывается внимание.
- Поиск по небольшим кусочкам текста работает и быстрее, и точнее.

Преобразование чанков в эмбединги

- Задаётся специальная модель, которая отображает чанк в эмбединг.
- Выбор модели зависит от:
 - Языка или языков используемых в тексте;
 - Предметной области;
 - Качества работы;
 - Доступных ресурсов;
 - Требований к скорости работы.
- Выбирать модель следует по конкретной задаче!

06 { Поиск (retrieval)

Поиск

- Получаем эмбединг запроса. Используем ту же самую модель, которую использовали для наполнения векторной БД!
- Поиск осуществляет векторная БД. Какие параметры можно задать:
 - Эмбединг запроса.
 - Максимальное количество результатов.
 - Параметры фильтрации по метаданным.
 - Функцию расстояния между векторами. Иногда её можно задать, но не в самом запросе, а при создании БД и/или при построении индекса.
- Обычно в качестве функции расстояния используют косинусное сходство – это косинус угла между двумя векторами. Сейчас обычно вектора нормируют к длине 1, поэтому косинусное сходство эквивалентно скалярному произведению. Чем выше значение этой функции, тем более схожи вектора, при значении 1.0 они идентичны.

07 { Генерация ответа

Как сгенерировать ответ?

- Для генерации ответа используется LLM.
- LLM передаётся системный промпт, в котором указывается её роль и указание использовать для ответа переданный ей контекст.
- Найденные чанки передаются в системный промпт как контекст.
- Далее, отдельным сообщением передаётся запрос пользователя.
- Пример:
 - SYSTEM: Ты эксперт в области ML. Отвечай на вопрос пользователя только используя контекст.
 - CONTEXT: Последняя версия Qwen: 3.6.
 - User: Какая последняя версия моделей Qwen?

08



Оценка качества работы
RAG

Зачем оценивать качество работы RAG?

- Для того, чтобы понимать насколько хорошо он работает, где в нём проблемы и обеспечивается ли достаточное качество работы для пользователей.
- Без оценки качества вы не сможете понять:
 - Достаточно ли у LLM контекста для ответа.
 - Находит ли поиск нужные чанки.
 - Не занимается ли LLM выдумыванием ответа, если его нет в найденной информации.
- Без оценивания качества работы вы слепы!

Как можно оценивать?

- RAG можно оценивать как сервис: время ожидания пользователя, затраты ресурсов и так далее. Это очень важно, но мы не будем сегодня рассматривать эту тему.
- RAG можно оценивать по качеству работы в целом и покомпонентно. Про это сегодня и поговорим.

Ключевые метрики работы RAG

- Популярным подходом конструирования метрик является LLM-as-judge (LLM как судья). В этом подходе в качестве функции оценки выступает LLM.
- Faithfulness. Показывает, насколько ответ верен переданному контексту. Она оценивает, насколько ответ от LLM основан только на переданном ей контексте, а не на внутренних знаниях модели.
- ContextRelevance. Показывает, насколько найденный чанк релевантен запросу.
- AnswerRelevancy. Показывает, насколько ответ релевантен запросу. Она оценивает семантическое сходство между запросом и ответом. Надо иметь в виду, что низкое значение этой метрики может быть нормальным по разным объективным причинам (различается стиль запроса и ответа, сильно различаются формулировки и т.д.).
- Есть и другие метрики. Выбирайте те, которые будут наиболее полезны именно в вашем проекте.

09



Прокачиваю наш RAG:
чанкинг

Что важно знать про чанкинг?

- Когда строим алгоритм разбиения документов на кусочки (чанки), мы преследуем цель сохранить смысловой контекст каждого кусочка, но при этом не перегрузить LLM, которую мы используем в нашем RAG.
- Мы имеем следующие проблемы:
 - Слишком короткий кусочек: мы можем потерять связь с логикой документа, соответствующим заголовком и разделом.
 - Слишком длинный кусочек: LLM начинает терять детали.
- Оптимальная длина чанка зависит от ваших задач и документов, например, в технической документации распространена длина в 300-500 токенов.
- Так же важно учитывать, сколько токенов на вход может принять ваш эмбеддер.

Как ещё можно сделать чанкинг?

- Разбиение фиксированного размера с наложениями друг на друга (overlap). Берём N символов/токенов, в следующем чанке начинаем с K последних символов предыдущего. Перекрытие очень важно, поскольку позволяет сохранить контекст на границах чанков. Может помочь во всяких длинных документах и логах.
- Разбиение по абзацам. Может быть достаточным в простых и коротких документах.
- Разбиение по структуре. Разбиваем по схеме заголовков + текст за ним. Может подойти в работе с документацией и Wiki системами.
- Иерархический чанкинг – смесь разбиения по структуре и разбиения фиксированного размера с наложением.
- Семантический чанкинг (с помощью эмбеддера) – делим текст там, где смысл резко меняется.

Метаданные чанков

- Полезно каждому чанку добавлять различные метаданные.
- В чём метаданные могут помочь:
 - Отладка: по метаданным можно понять, откуда появился чанк в результатах поиска.
 - Фильтрация: можно ограничивать области поиска определёнными разделами, или, например, свежестью данных.
 - Безопасность: можно искать только ту информацию, к которой имеет доступ пользователь.

10



Прокачиваем наш RAG:
поиск

Reranking

- Векторный поиск по сходству эмбеддингов отличная штука, но у неё есть проблема – сортировка по близости векторов не всегда совпадает с семантической релевантностью для пользователя.
- Решить эту проблему можно с помощью реранкера.
- Реранкер это модель, которая принимает на вход запрос и чанк, и оценивает релевантность чанка запросу.
- По этой оценке можно пересортировать найденные чанки и использовать для генерации ответа только top-k чанков, где k меньше чем максимальное количество найденных векторным поиском чанков.
- В такой схеме рекомендуется искать векторным поиском не менее 3 чанков, на практике им ищут 10-20 кандидатов, после реранкинга выбирают top-3 – top-5 чанков.
- По сути, это классический 2х стадийный подход, давно используемый в машинном обучении. Сначала, первой моделью мы получаем кандидатов с максимальной полнотой, но не очень высокой точностью, потом отфильтровываем результат более сильной моделью.

Когда стоит задействовать reranking?

- Вы ищете векторным поиском большое количество чанков (не меньше трёх). На практике часто ищут 10-20 кандидатов.
- Очень важна точность ответов.
- У вас уже хорошо настроен базовый пайплайн (хороший чанкинг, фильтрация по метаданным, используются достаточно хорошие модели и так далее).
- У вас есть запас по времени и ресурсам на ответ пользователю, поскольку reranking замедляет скорость ответа пользователю и потребляет дополнительные вычислительные ресурсы.

Когда не стоит задействовать reranking?

- Вы делаете прототип – применять reranking ещё рано, сначала надо добиться хорошей работы от базовых компонентов системы.
- Точность не критична.
- Нет запаса по времени на ответ пользователю и/или не хватает вычислительных ресурсов.

Что нужно учитывать при применении reranking?

- У большинства моделей reranking небольшое окно контекста (примерно 512 токенов). Ваши чанки могут не пролезать в контекст.
- Что можно с этим сделать:
 - Можно делить документы на чанки так, чтобы они пролезали в окно контекста реранкера.
 - Если чанки для векторного поиска нельзя сделать короче – перед подачей в ранкер обрезайте их.

Можно ли сделать reranking лучше?

- Да, можно, с помощью дообучения на своих данных. Для этого нужно следующее:
 - Вы измеряете качество работы RAG.
 - Ваши данные достаточно специфичны для того, чтобы уже обученные реранкеры систематически ошибались на ваших запросах и данных.
 - У вас есть или вы можете получить датасет пар «запрос – релевантный чанк» с ручной разметкой размером хотя бы 500 примеров.
- Но на старте этого делать не стоит, потому что вы пока не умеете замерять качество, вы не знаете насколько хорошо работают готовые реранкеры и, скорее всего, у вас нет датасета для дообучения.

Гибридный поиск

- К сожалению, семантический поиск с помощью эмбедингов может находить не все релеватные чанки.
- Помочь может лексический поиск с помощью, например, алгоритма BM25.
- Объединение семантического поиска с лексическим называется гибридным поиском.
- Объединять результаты обоих поисков можно по разному, например, с помощью переранжирования реранкером.

Multi-query

- Можно дорабатывать не только сам процесс поиска, но и запрос пользователя.
- Одним из таких подходов является multi-query.
- Применяется на коротких запросах (примерно 3-4 слова максимум).
- Можно расширять запрос с помощью шаблонов или с помощью LLM.
- Вместо одного запроса появляется несколько.

11



Прокачиваем наш RAG:
guardrails

Что это такое?

- Guardrails в RAGax это не отдельный модуль, а стиль работы со всеми компонентами разрабатываемой системы. При её разработке вы постоянно должны думать о том, где что-то может пойти не так, где вас могут сломать, где пользователи могут получить данные, к которым у них доступа нет и так далее.
- Важной особенностью RAGов является то, что LLM не гарантирует результата.
- Примеры угроз:
 - Векторный троян с промпт-инъекцией на этапе индексации.
 - Промпт-инъекция в пользовательском запросе.
 - Получение доступа к данным пользователем, у которого на это нет прав.
 - Утечка секретов.
 - Галлюцинации в ответе.

Как бороться с проблемами?

- Для начала, нужно понять, какие проблемы возможны в вашем случае.
- После выявления проблем нужно подумать, насколько они критичны и какие методы борьбы с ними будут достаточны.
- Основные способы построения защитных механизмов:
 - Алгоритмические.
 - Регулярные выражения.
 - Классический ML.
 - Различные методы из NLP.
 - LLM-as-judge.



Удачи и большое спасибо
за внимание!

Степан Кудин

email: kudin.stepan@yandex.ru

tg: Kalimdor